Capítulo 4 · Índices en MongoDB

Cuaderno de prácticas — laboratorio guiado desde cero

Destinatarios: alumnos que todavía no han trabajado con `mongosh` ni han creado un índice.
Entorno: Docker Desktop/Engine con Docker Compose, repositorio MONGODB y MongoDB Community 8.0.
Regla de trabajo: cuando un paso diga «Terminal», se ejecuta en PowerShell, Bash o Windows Terminal; cuando diga «mongosh», se escribe dentro del intérprete de MongoDB.

Objetivo general. Al terminar tendrás que poder demostrar con tus propias palabras por qué una búsqueda examina muchos documentos sin índice y por qué un índice apropiado cambia ese trabajo. También crearás índices compuestos y especializados, entenderás errores de unicidad y observarás la caducidad TTL.

No memorices los comandos. Lee primero la pregunta que resolverás, ejecuta el paso y compara lo ocurrido con la explicación. Si algo no coincide, utiliza las comprobaciones indicadas. Todos los datos del curso son ficticios.

---

0. Lo que vas a encontrar en pantalla

Terminal del sistema

Una terminal es una ventana desde la que se ejecutan programas. En Windows puedes usar **PowerShell** o Windows Terminal. En macOS/Linux, una terminal Bash o similar.

Por ejemplo, este comando pertenece a la **terminal**:

```
docker compose ps
```

Si lo escribes dentro de `mongosh`, obtendrás un error porque MongoDB no interpreta órdenes de Docker.

Intérprete mongosh

`mongosh` es el programa que permite dar instrucciones a MongoDB. Una vez dentro se admiten expresiones JavaScript y operaciones como:

```
db.getName()
db.ventas.countDocuments({})
```

Si escribes `db.ventas.countDocuments({})` directamente en PowerShell, no funcionará: todavía no te has conectado a MongoDB.

Cómo reconocer dónde estás

En PowerShell verás normalmente algo parecido a `PS C:\...>`. En `mongosh` aparece el nombre de la base actual, por ejemplo `curso_indices>`. La apariencia depende de las versiones; no hace falta que los símbolos coincidan exactamente.

En estas prácticas ofrecemos dos maneras de operar: **ejecutar un script del repositorio desde la terminal** o **entrar en `mongosh` y escribir consultas**. Cada vez indicaremos cuál corresponde.

---

1. Preparar el proyecto sin asumir nada

Qué haremos y por qué

Antes de crear índices necesitamos un servidor MongoDB disponible. El repositorio incluye `compose.yaml`, un archivo que describe los contenedores Docker que formarán el laboratorio. Docker permite ejecutar la misma versión de MongoDB sin instalar a mano sus componentes en tu sistema.

Necesitas Docker Desktop o Docker Engine con Compose v2, acceso para descargar la imagen y espacio libre. La primera descarga puede tardar; no es un síntoma de error.

Paso 1. Obtener los archivos

Abre el repositorio. Puedes pulsar **Code → Download ZIP**, extraerlo y abrir una terminal **dentro de la carpeta extraída**. Si ya utilizas Git, también puedes clonarlo:

```
git clone https://github.com/jaimecopilot/MONGODB.git
cd MONGODB
```

Importante: la carpeta elegida debe contener `compose.yaml`. Para verificarlo, usa `dir` en PowerShell o `ls` en Bash. Si no aparece, probablemente estás en una carpeta equivocada.

Paso 2. Preparar la configuración

El repositorio proporciona `.env.example`, una plantilla de valores de laboratorio. Docker Compose requiere una variable `MONGO_ADMIN_PASSWORD` para el servicio de seguridad, incluso aunque en este primer laboratorio solo iniciemos el servicio sin autenticación.

En **PowerShell**:

```
Copy-Item .env.example .env
notepad .env
```

En **Bash**:

```
cp .env.example .env
# Abre .env con tu editor de texto preferido
```

Reemplaza las contraseñas de demostración por otras **creadas únicamente para este laboratorio**. Guarda el archivo. Nunca lo publiques en GitHub: es privado y está excluido por `.gitignore`.

Comprueba ahora: existen `compose.yaml` y `.env` en la misma carpeta. El archivo `.env` no contiene contraseñas personales ni datos de un servidor verdadero.

Paso 3. Comprobar Docker

En la **terminal**:

```
docker --version
docker compose version
```

El programa responderá con sus números de versión. Si aparece «docker no se reconoce» o «command not found», Docker no está instalado o la terminal no lo encuentra. Si el comando existe pero aparece «Cannot connect to the Docker daemon», Docker Desktop/Engine probablemente no está iniciado.

Paso 4. Encender únicamente MongoDB de índices

```
docker compose up -d --wait mongo-indices
```

- `up` crea o inicia el servicio.
- `-d` lo deja funcionando sin ocupar la terminal.
- `--wait` espera a que su comprobación de salud sea satisfactoria.
- `mongo-indices` es el nombre del servicio en `compose.yaml`.

Después ejecuta:

```
docker compose ps
```

Busca `mongo-indices` y un estado equivalente a *healthy*. El puerto externo se publica en `127.0.0.1:27017`: solo está disponible en tu propio equipo, no para cualquier ordenador de la red. Este servicio no usa autenticación y por eso **solo albergará datos ficticios**.

Si el servicio no arranca: inspecciona `docker compose logs --tail=80 mongo-indices`; comprueba que otro MongoDB no esté ocupando el puerto 27017; verifica que Docker dispone de memoria suficiente.

Paso 5. Primera conexión, con resultado visible

```
docker compose exec mongo-indices mongosh
```

La terminal se convertirá en una sesión interactiva de `mongosh`. Escribe:

```
db.adminCommand({ping: 1})
```

Deberías ver un objeto con `ok: 1` (puede contener otros datos). **Ese resultado indica que el servidor responde**, no que hayas creado todavía ventas ni índices.

Para salir de `mongosh`:

```
exit
```

Si el comando `exit` no resulta cómodo en tu terminal, `.exit` es otra posibilidad aceptada por el intérprete.

Comprobación de aprendizaje: ¿qué comprobaste con `ping`? Que la conexión funciona. ¿Qué no comprobaste? Que existe la colección `ventas` ni que sus consultas sean rápidas.

---

2. Cargar ventas ficticias y comprender qué hemos creado

El problema

Para comparar índices hacen falta suficientes documentos. Con diez ventas, una consulta sin índice quizá parezca instantánea; además, puede ser racional recorrerlas todas. En este ejercicio generaremos **200.000 ventas sintéticas**, sin información real de personas.

Paso 1. Ejecutar el generador

De vuelta en la **terminal**, ejecuta:

```
docker compose exec -T -e VENTAS=200000 mongo-indices mongosh --quiet --file /lab/
01_cargar_ventas.js
```

Lee este comando por partes:

- `docker compose exec` ejecuta otro programa dentro de un contenedor que ya está encendido.
- `-T` desactiva la asignación de un terminal interactivo, adecuada para un script.
- `-e VENTAS=200000` indica al generador cuántos documentos crear.
- `mongo-indices` designa el servicio.
- `mongosh --quiet --file` ejecuta un archivo de comandos sin abrir una sesión para escribir a mano.
- `/lab/01_cargar_ventas.js` es la **ruta dentro del contenedor**; gracias al volumen de Compose corresponde a un script del repositorio.

El generador informa del progreso por lotes y debe finalizar con un mensaje de confirmación. Si utilizas un equipo con pocos recursos, evita abrir varias cargas simultáneamente.

Advertencia crucial: volver a ejecutar la carga borra y recrea `curso_indices.ventas`; **desaparecen también sus índices secundarios**. Hazlo ahora, antes de empezar a medir; más tarde solo cuando quieras reiniciar el experimento.

Paso 2. Entrar en mongosh y mirar la colección

```
docker compose exec mongo-indices mongosh
```

Dentro de `mongosh`, escribe:

```
use curso_indices
db.ventas.countDocuments({})
```

Esperamos `200000`. La colección contiene 200.000 ventas.

Veamos solo **un** documento, para no inundar la pantalla:

```
db.ventas.findOne()
```

Localiza `numero`, `clienteId`, `estado`, `fecha`, `importe` y `etiquetas`. El contenido exacto depende de qué documento devuelva MongoDB; no necesitas encontrar un valor concreto en esta primera inspección.

Paso 3. Preguntar qué índices existen

```
db.ventas.getIndexes()
```

Al haber reiniciado la colección, el índice esperado es `_id_`. Observa que MongoDB creó ese índice automáticamente. **Todavía no hemos creado un índice para `numero`.**

Si aparecen muchos otros índices, probablemente no has reiniciado la colección o estás mirando otra base. Comprueba `db.getName()` y repite la carga antes de continuar.

Paso 4. Una pregunta sencilla

```
db.ventas.findOne({numero: 199998})
```

Deberías obtener el documento correspondiente al número pedido. Este es el primer contacto con el filtro: las llaves expresan «el campo `numero` es igual al valor dado». Aunque el resultado aparezca rápido, **no sabemos cuánto trabajo hizo MongoDB**.

No pases a la siguiente práctica hasta poder decir: el conjunto es una colección; cada venta es un documento; `countDocuments({})` cuenta todos los documentos; `getIndexes()` enumera índices; `findOne()` busca un resultado.

---

3. Medir una consulta SIN índice: COLLSCAN

La pregunta que queremos contestar

¿Cuántos documentos debe inspeccionar MongoDB para localizar una venta cercana al final de la colección si no existe índice sobre `numero`?

No mediremos simplemente «si aparece la venta»: eso ya lo sabemos. Mediremos el **trabajo**.

Paso 1. Confirmar que seguimos sin índice secundario

Dentro de `mongosh`:

```
db.ventas.getIndexes()
```

Si solo aparece `_id_`, tenemos una línea de base válida.

Paso 2. Ejecutar explain() en la misma consulta

```
db.ventas.find({numero: 199998}).explain("executionStats")
```

Este resultado suele ser largo. Busca:

- `nReturned`: cuántas ventas devolvió.
- `totalDocsExamined`: cuántos documentos necesitó examinar.
- `totalKeysExamined`: cuántas claves de índice examinó.
- `COLLSCAN`: una etapa que recorre la colección.
- `executionTimeMillis`: duración observada para esa ejecución.

Si ves muchas llaves y propiedades, no es un error: `explain()` devuelve información técnica estructurada.

Paso 3. Obtener una salida resumida

Copia este código **dentro de mongosh**:

```
const sinIndice = db.ventas
  .find({numero: 199998})
  .explain("executionStats");

printjson({
  devueltos: sinIndice.executionStats.nReturned,
  documentosExaminados: sinIndice.executionStats.totalDocsExamined,
  clavesExaminadas: sinIndice.executionStats.totalKeysExamined,
  milisegundos: sinIndice.executionStats.executionTimeMillis
});
```

Es un objeto de cuatro propiedades. Si más adelante repites el bloque en la misma sesión, puedes recibir un error por declarar dos veces `const sinIndice`; usa otro nombre, sustituye `const` por `let` al comienzo de una sesión nueva o entra de nuevo al intérprete.

Con 200.000 ventas esperamos un número alto en `documentosExaminados`, próximo al total. No impongas que coincida exactamente si has modificado los datos.

Paso 4. Ejecutar el script equivalente

Sal de `mongosh` con `exit`. En la terminal:

```
docker compose exec -T mongo-indices mongosh --quiet --file /lab/02_antes.js
```

Este script comprueba que no haya índices secundarios y resume la consulta. **Si falla indicando que ya existen índices**, es una protección didáctica: vuelve al paso de carga `01_cargar_ventas.js` y repite la comparación desde cero.

La explicación que debes escribir en tu cuaderno

«La consulta devuelve __ documento(s), pero el servidor ha examinado __ documentos; como todavía no hay índice sobre `numero`, el plan necesita recorrer la colección. El tiempo observado fue de __ ms en mi máquina; no significa que en todas las máquinas tarde lo mismo».

Error habitual

Si `nReturned` es 0, comprueba que hayas cargado al menos 199.998 ventas. También revisa que hayas buscado un número `199998` y no el texto `"199998"`. Los tipos de datos son importantes.

---

4. Crear el índice simple y repetir la MISMA consulta

Esta práctica es el corazón del capítulo. Si solo puedes enseñar una demostración de indexación, debe ser esta.

Paso 1. Crear un índice sobre numero

Dentro de `mongosh`:

```
db.ventas.createIndex({numero: 1}, {name: "ix_numero", unique: true})
```

- `numero` es el campo indexado.
- `1` indica dirección ascendente.
- `name` asigna un nombre fácil de reconocer.
- `unique: true` establece que dos ventas no podrán compartir el mismo número.

El resultado deberá contener el nombre del índice. Si dice que existe un índice incompatible, revisa `getIndexes()`: quizá ejecutaste antes `03_crear_indices.js` o diseñaste otro índice sobre las mismas claves con opciones diferentes. Para el itinerario limpio, reinicia la colección y sigue el orden.

Paso 2. Comprobar que MongoDB lo reconoce

```
db.ventas.getIndexes()
```

Ahora deben aparecer al menos dos índices: `_id_` e `ix_numero`. Fíjate en la diferencia entre el nombre del índice y el campo que utiliza.

Paso 3. Repetir la búsqueda, SIN cambiar el filtro

```
db.ventas.find({numero: 199998}).explain("executionStats")
```

Compara el resultado con el de la práctica 3. Normalmente verás una etapa `IXSCAN`, quizá incluida bajo `FETCH`. Lo relevante es que la consulta por ese número examina muchas menos entradas y documentos.

Para obtener contadores:

```
const conIndice = db.ventas.find({numero: 199998}).explain("executionStats");
printjson({
  devueltos: conIndice.executionStats.nReturned,
  documentosExaminados: conIndice.executionStats.totalDocsExamined,
  clavesExaminadas: conIndice.executionStats.totalKeysExamined,
  milisegundos: conIndice.executionStats.executionTimeMillis
});
```

Paso 4. Completar una comparación

Medida	Antes	Después
`nReturned`	Anótalo	Anótalo
`totalDocsExamined`	Anótalo	Anótalo
`totalKeysExamined`	Anótalo	Anótalo
`executionTimeMillis`	Anótalo	Anótalo
Plan (`COLLSCAN` o `IXSCAN`)	Anótalo	Anótalo

La conclusión correcta: tenemos la misma pregunta y el mismo resultado, pero el motor utiliza un acceso más adecuado. El tiempo registrado es una observación y no una garantía universal.

Paso 5. Recuperar la secuencia preparada de la clase

Si has seguido únicamente comandos manuales, puedes avanzar así. Si prefieres que el repositorio prepare todos los índices siguientes, sal de `mongosh` y ejecuta:

```
docker compose exec -T mongo-indices mongosh --quiet --file /lab/03_crear_indices.js
docker compose exec -T mongo-indices mongosh --quiet --file /lab/04_despues.js
```

El primer script crea `ix_numero` y otros índices necesarios para las prácticas posteriores. Como ya creaste `ix_numero` con el mismo diseño y nombre, MongoDB puede reconocerlo como existente. El segundo script ejecuta una nueva medición.

Si aparece un conflicto de opciones, reinicia con `01_cargar_ventas.js` y usa la ruta totalmente automatizada `02_antes.js → 03_crear_indices.js → 04_despues.js`. No alteres la colección hasta que termine la comparación.

Pregunta de comprensión

Si `documentosExaminados` pasó de 200.000 a 1, ¿se redujo también el número de ventas almacenadas? No. Cambió el **trabajo necesario para encontrarlas**.

---

5. Dos campos en el mismo índice: cliente y fecha

Situación real

Una persona consulta sus últimas cinco compras. El personal no quiere todas las ventas de todas las personas, sino las de **un cliente concreto, ordenadas desde la más reciente**.

Aquí hay dos operaciones: filtro por cliente y ordenación por fecha.

Paso 1. Entender la consulta antes de crear nada

Dentro de `mongosh`:

```
db.ventas
  .find({clienteId: 809})
  .sort({fecha: -1})
  .limit(5)
```

- `find` selecciona ventas cuyo `clienteId` es 809.
- `sort({fecha:-1})` pide fechas de mayor a menor.
- `limit(5)` solicita como máximo cinco resultados.

Verás hasta cinco ventas, si el cliente existe. No te preocupes por los importes: el objetivo es el patrón de acceso.

Paso 2. Diseñar un índice compuesto

```
db.ventas.createIndex(
  {clienteId: 1, fecha: -1},
  {name: "ix_cliente_fecha"}
)
```

No creamos «dos índices». Creamos una sola estructura cuyo primer criterio es `clienteId` y, dentro de él, `fecha` descendente.

Paso 3. Comprobar el plan

```
db.ventas
  .find({clienteId: 809})
  .sort({fecha: -1})
  .limit(5)
  .explain("executionStats")
```

¿Aparece `IXSCAN`? ¿Aparece una etapa `SORT` independiente? Mira el plan completo.

Importante: si previamente ejecutaste `03_crear_indices.js`, ya habrá índices alternativos sobre campos relacionados. MongoDB podría elegir otro. El propósito es **razonar la propuesta**, no obligar al motor a un plan por memoria.

Paso 4. Cambiar solo la pregunta y observar qué pasa

Prueba:

```
db.ventas.find({fecha: {$gte: ISODate("2025-07-01")}}).limit(5)
```

Ahora no filtramos por cliente. El índice compuesto comienza por `clienteId` y quizá no ayude de la misma manera. Esa es la razón práctica por la que el orden de campos importa.

¿Qué es un prefijo izquierdo?

Si las claves están organizadas primero por cliente y después por fecha, es fácil encontrar las de un cliente. No podemos esperar el mismo acceso a «todas las ventas de julio» si no se conoce antes el cliente. Ese razonamiento es más valioso que memorizar las siglas `ESR`.

Si algo falla

Si un campo `fecha` contiene texto en lugar de fecha, el rango podría ser distinto del esperado. Inspecciona `findOne()` y busca que aparezca un tipo de fecha. Si no estás en `curso_indices`, vuelve con `use curso_indices`.

---

6. ESR: igualdad, ordenación y rango en una consulta

Caso de negocio

El responsable de una tienda necesita las cinco ventas recientes cuyo importe cumple determinada condición.

```
db.ventas.find({
  tienda: "Madrid",
  fecha: {$gte: ISODate("2025-07-01")},
  importe: {$gte: 100}
}).sort({fecha: -1}).limit(5)
```

Ahora combinamos igualdad por tienda, un rango de fechas, otro rango por importe y una ordenación. Esta consulta no se optimiza necesariamente con el primer índice que imaginemos.

Paso 1. Separar las piezas

Escribe en papel:

- Igualdad: `tienda` es `"Madrid"`.
- Rango: `fecha` a partir del 1 de julio.
- Rango: `importe` de 100 o más.
- Ordenación: `fecha` descendente.
- Límite: cinco documentos.

¿Por qué importa esta lista? Porque un índice compuesto se diseña para **consultas reales**, no para almacenar todos los campos al azar.

Paso 2. Crear un candidato y medir

```
db.ventas.createIndex(
  {tienda: 1, fecha: -1, importe: 1},
  {name: "ix_tienda_fecha_importe"}
)
```

Ejecuta el `explain("executionStats")` de la misma consulta. Busca `totalKeysExamined`, `totalDocsExamined` y si MongoDB necesitó una ordenación adicional.

Paso 3. Interpretar sin dogmatismo

La sigla **ESR** ayuda a razonar igualdad, sort y range, pero aquí tenemos más de una condición de rango. No existe una receta que garantice el mejor plan en todo caso. Podrían influir la cantidad de ventas de Madrid, la distribución de importes y la fecha del límite.

Resultado que debes registrar: el nombre del índice elegido, documentos examinados, claves examinadas y si el plan contiene `SORT`.

Diagnóstico

Si creas el índice y el planificador usa otro, no quiere decir automáticamente que el servidor esté equivocado. Compara los contadores, revisa qué otros índices existen y explica cuál sería la ventaja de cada candidato.

---

7. Arrays e índice multikey: buscar una etiqueta

Qué tiene de particular el dato

Una venta puede contener varias etiquetas:

```
{numero: 1001, etiquetas: ["venta", "web", "normal"]}
```

Queremos consultar solo las ventas que contienen `"web"`.

Paso 1. Busca sin pensar todavía en índices

```
db.ventas.find({etiquetas: "web"}).limit(3)
```

Observa que el filtro utiliza un texto, pero el campo guardado es una lista. MongoDB interpreta la consulta como «este array contiene el valor». No busca un documento cuyo campo sea literalmente solo el texto `"web"`.

Paso 2. Crear el índice

```
db.ventas.createIndex({etiquetas: 1}, {name: "ix_etiquetas"})
```

Este índice es **multikey** cuando se indexa un campo de tipo array. MongoDB mantiene entradas para los elementos indexados; el documento de venta no se duplica en la colección.

Paso 3. Forzar una observación controlada del índice

```
db.ventas.find({etiquetas: "web"})
  .hint("ix_etiquetas")
  .limit(3)
  .explain("executionStats")
```

`hint` indica un índice concreto para este experimento. Lo usamos **para estudiar su funcionamiento**, no como recomendación de producción: imponer un índice que no conviene puede empeorar el rendimiento.

Paso 4. Encontrar un error conceptual

Una venta con `["venta","web","normal"]` coincide con `{etiquetas:"web"}`, pero su array no es exactamente igual a `["web"]`. Explica por qué es peligroso confundir ambas ideas.

Si no aparece el índice: ejecuta `getIndexes()`; verifica que `ix_etiquetas` existe y que estás usando la colección correcta.

---

8. Buscar palabras en descripciones

La diferencia entre igualdad y texto completo

En algunas ventas la descripción es `"reclamacion urgente del cliente"`. La igualdad:

```
db.ventas.find({descripcion: "reclamacion"})
```

no equivale a «contiene la palabra reclamación». Para buscar términos utilizaremos el mecanismo de texto clásico.

Paso 1. Crear el índice de texto

```
db.ventas.createIndex(
  {descripcion: "text"},
  {name: "ix_descripcion_texto", default_language: "spanish"}
)
```

En el banco del curso suele existir ya un índice con ese nombre si ejecutaste `03_crear_indices.js`. En ese caso, compruébalo con `getIndexes()` en vez de crear otro incompatible: una colección tiene restricciones sobre índices `text`.

Paso 2. Consultar un término

```
db.ventas.find({$text: {$search: "reclamacion"}}).limit(5)
```

Deberías encontrar ventas cuya descripción fue indexada con ese término. La palabra exacta, el idioma y las reglas de normalización pueden afectar las coincidencias.

Paso 3. Verificar que no es lo mismo que igualdad

Ejecuta ahora:

```
db.ventas.find({descripcion: "reclamacion"}).limit(5)
```

Compara resultados. Es posible que no aparezca ninguno: la descripción completa es una frase y no una palabra aislada.

Paso 4. Pensar como diseñador

¿Serviría el índice de texto para encontrar «las ventas de 100 a 200 euros»? No. ¿Serviría uno de importe para buscar términos en una frase? Tampoco. Los índices responden a *clases distintas de consultas*.

Si la consulta `$text` dice que no hay índice de texto: ejecuta primero `03_crear_indices.js` o crea el índice del paso 1 y comprueba que su construcción ha terminado.

9. Puntos geográficos e índices hashed: dos experimentos distintos

Parte A. Qué es un punto geográfico

En el curso el campo `ubicacion` contiene objetos GeoJSON. Un ejemplo de coordenadas es:

```
{type: "Point", coordinates: [-3.7038, 40.4168]}
```

El primer número es **longitud** y el segundo **latitud**. Son cosas distintas: invertirlos cambia el lugar.

Paso A1. Construir el índice

```
db.ventas.createIndex({ubicacion: "2dsphere"}, {name: "ix_ubicacion"})
```

Paso A2. Buscar próximos a una coordenada

```
db.ventas.find({
  ubicacion: {
    $near: {
      $geometry: {type: "Point", coordinates: [-3.7038, 40.4168]},
      $maxDistance: 20000
    }
  }
}).limit(3)
```

Con geometrías GeoJSON y `2dsphere`, `$maxDistance` se interpreta aquí en **metros**. La salida depende de las posiciones ficticias generadas; esta práctica se centra en la forma de la consulta.

Parte B. Qué significa hashed

El campo `idGrupo` identifica un grupo ficticio. Un índice hashed transforma sus claves para facilitar accesos por igualdad en ciertos contextos.

```
db.ventas.createIndex({idGrupo: "hashed"}, {name: "ix_grupo_hash"})
db.ventas.find({idGrupo: 809}).hint("ix_grupo_hash").limit(3)
```

La conclusión importante: un índice hashed puede resultar útil para igualdad, pero no conserva la ordenación natural necesaria para un rango de números. No uses `hashed` porque «parece más moderno» ni porque creas que hace más rápida toda búsqueda.

Parte C. Script conjunto

Sal de `mongosh` y ejecuta desde la terminal:

```
docker compose exec -T mongo-indices mongosh --quiet --file /lab/05_tipos.js
```

El script comprueba búsquedas sobre etiquetas, texto, localización e igualdad hashed. Si falla la creación de un índice, revisa si ya existe otro con diseño u opciones incompatibles. **No borres índices a ciegas**: `getIndexes()` debe ser la primera comprobación.

---

10. Unique: comprobar una regla mediante un error

Por qué vamos a provocar un fallo

Una base de datos no solo debe encontrar documentos con rapidez; también debe impedir estados de datos no válidos. En este ejercicio usaremos una colección **aislada**, `unicidad_demo`, para aprender a rechazar un correo duplicado.

Paso 1. Crear la colección experimental

Dentro de `mongosh`:

```
use curso_indices
db.unicidad_demo.drop()
db.unicidad_demo.createIndex({email: 1}, {unique: true})
```

`drop()` elimina la colección de demostración si existía. **No lo ejecutes en colecciones con datos reales**.

Paso 2. Insertar el primer documento

```
db.unicidad_demo.insertOne({email: "alumno@ejemplo.test"})
```

El resultado incluirá `acknowledged: true` y un identificador `insertedId`. Esto quiere decir que MongoDB aceptó la inserción.

Paso 3. Intentar duplicarlo

```
db.unicidad_demo.insertOne({email: "alumno@ejemplo.test"})
```

Ahora esperamos un error `E11000 duplicate key`. **Este error es la respuesta correcta del ejercicio.** No se soluciona reiniciando MongoDB. Significa que el índice ha protegido la regla que declaramos.

Paso 4. Recuperarse y comprobar los datos

```
db.unicidad_demo.countDocuments({})
db.unicidad_demo.find()
```

El contador debe mostrar **1**: el duplicado no se insertó.

Interpretación

Si el segundo `insertOne` no falla, probablemente no creaste el índice `unique` sobre la colección correcta o eliminaste la colección después de crearlo. Revisa `getIndexes()`.

El índice único sobre correo es apropiado aquí solo porque la regla del ejercicio dice que cada correo debe aparecer una vez. En otras aplicaciones dos usuarios podrían legítimamente compartir dirección, y esa regla tendría que revisarse.

---

11. TTL: observar que un documento caduque sin borrarlo a mano

El caso

Una sesión temporal debe dejar de almacenarse al expirar. En vez de programar manualmente un borrado de sesiones antiguas, MongoDB puede usar un índice TTL sobre un campo de fecha.

Paso 1. Crear datos de prueba

Dentro de `mongosh`:

```
use curso_indices
db.sesiones_demo.drop()

db.sesiones_demo.createIndex(
  {expiraEn: 1},
  {expireAfterSeconds: 0}
)

db.sesiones_demo.insertMany([
  {_id: "expirado", expiraEn: new Date(Date.now() - 60000)},
  {_id: "vigente", expiraEn: new Date(Date.now() + 3600000)}
])
```

El primer documento expiró hace aproximadamente un minuto; el segundo expirará dentro de aproximadamente una hora. Ambos tienen un campo `Date` verdadero.

Paso 2. Observar qué existe ahora

```
db.sesiones_demo.find()
```

Es posible que el expirado ya haya desaparecido o que todavía esté presente. **Ambas observaciones son compatibles con TTL** porque el borrado se hace de forma asíncrona.

Paso 3. Dar tiempo al monitor sin confundir conceptos

Vuelve a ejecutar:

```
db.sesiones_demo.find()
```

Cuando actúe el monitor, `"expirado"` desaparecerá, mientras que `"vigente"` deberá permanecer.

En este contenedor docente el monitor TTL se ejecuta con una frecuencia de comprobación acelerada para facilitar el experimento. En un sistema real no debes depender de que la eliminación se haga en el milisegundo exacto de expiración.

Paso 4. Ejecutar la verificación reproducible

Desde la **terminal**:

```
docker compose exec -T mongo-indices mongosh --quiet --file /lab/06_unique_ttl.js
docker compose exec -T mongo-indices mongosh --quiet --file /lab/07_ver_ttl.js
```

El primer script crea el caso y el segundo espera, con límite temporal, a que el monitor elimine el documento expirado. Debes ver confirmación del borrado y de que el vigente sigue existiendo.

Si falla

Primero comprueba que `expiraEn` contiene objetos de fecha y no textos. Después inspecciona `getIndexes()` para encontrar `expireAfterSeconds: 0`. Si el servidor está muy cargado, el monitor puede tardar más de lo esperado. Un test con tiempo límite informa de una **observación no completada**; no demuestra por sí solo que MongoDB no admita TTL.

Pregunta crítica

¿Es seguro utilizar exclusivamente TTL para impedir que se use una sesión caducada? No: la aplicación debe comprobar también la fecha de expiración cuando decide si acepta una sesión, porque la eliminación física puede retrasarse.

---

12. Proyecto final: enseñar lo aprendido a otra persona

Esta última actividad es la evaluación de comprensión. No consiste en obtener diez mensajes «OK»; consiste en explicar **qué ocurrió y por qué**.

Parte 1. Elegir una consulta

Elige una de estas preguntas:

- «Buscar una venta por su número».
- «Ver las cinco ventas más recientes de un cliente».
- «Buscar una palabra en las descripciones».
- «Encontrar ventas etiquetadas como web».

Escribe qué campos utilizas, qué índice crees apropiado y por qué.

Parte 2. Comprobar con herramientas

En `mongosh` ejecuta la consulta y `explain("executionStats")`. Anota el número de documentos devueltos, examinados y claves visitadas. Escribe también qué etapa de acceso aparece.

Si decides comparar antes/después de un índice, recuerda que debes partir de una línea de base adecuada. Ejecutar otra vez `01_cargar_ventas.js` reinicia la colección y borra sus índices secundarios.

Parte 3. Construir una explicación

Redacta cinco frases sin emplear la palabra «rápido» como única justificación:

1. «La pregunta de negocio era...»
2. «La consulta sobre MongoDB fue...»
3. «El servidor eligió un plan que...»
4. «Después de diseñar el índice..., se observaron...»
5. «El índice tiene como contrapartida...»

Parte 4. Comprobar los recursos que dejas encendidos

En la **terminal**:

```
docker compose ps
```

Puedes apagar solo el servicio de índices sin destruir los datos:

```
docker compose stop mongo-indices
```

O eliminar todos los contenedores del proyecto (y, si añades `--volumes`, también sus volúmenes persistentes). No ejecutes el borrado de volúmenes sin comprender sus consecuencias.

---

13. Seleccionar columnas no es lo mismo que filtrar documentos

En esta actividad no vas a crear ningún índice. Vas a aprender a distinguir dos conceptos que suelen confundirse: **el filtro** decide qué documentos cumplen la condición; **la proyección** decide qué campos se muestran de esos documentos.

Dentro de `mongosh`, selecciona la base:

```
use curso_indices
db.ventas.find(
  {clienteId: 809},
  {_id: 0, numero: 1, importe: 1}
).limit(5)
```

Cómo leerlo: el primer objeto entre llaves es el filtro por cliente. El segundo dice «incluye el número y el importe, pero no muestres `_id`». No cambia qué ventas pertenecen al cliente: cambia los campos devueltos.

Repite sin proyección:

```
db.ventas.find({clienteId: 809}).limit(5)
```

Qué observar: en la segunda salida aparecen más campos. Es posible que el número de documentos devueltos siga siendo el mismo. Eso no significa que la consulta haya trabajado igual internamente.

Comprobación: ejecuta `.explain("executionStats")` para ambas consultas y compara `nReturned` y `totalDocsExamined`. ¿Son iguales? En determinadas condiciones pueden serlo, porque solo hemos modificado la forma de los resultados.

Si no ves registros: verifica que el campo se llama `clienteId`, que su valor es numérico y que la colección tiene datos cargados. Si `db.ventas.countDocuments({})` devuelve cero, vuelve al laboratorio de carga.

Lo que debes explicar: una proyección puede reducir datos enviados al cliente y participar en una consulta cubierta, pero una proyección por sí sola no crea un índice.

---

14. Consulta cubierta: demostrar que se leen claves y no documentos

Esta práctica responde a una pregunta más fina que «¿usó un índice?». Queremos saber si el servidor puede obtener una respuesta únicamente desde las entradas del índice.

Preparar un índice que contiene el filtro y la proyección

Dentro de `mongosh`:

```
db.ventas.createIndex(
  {clienteId: 1, importe: 1},
  {name: "ix_cobertura_cliente_importe"}
)
```

Ahora ejecuta:

```
db.ventas.find(
  {clienteId: 809},
  {_id: 0, clienteId: 1, importe: 1}
).hint("ix_cobertura_cliente_importe")
 .limit(5)
 .explain("executionStats")
```

Busca `totalDocsExamined`. En este patrón esperamos que pueda valer **0**: la respuesta está disponible en las claves del índice. El servidor podrá examinar varias claves y devolver cinco documentos lógicos sin tener que leer el documento completo.

¿Por qué escribimos `_id: 0`? Porque MongoDB incluiría normalmente `_id` en la salida y ese campo no forma parte del índice que estamos utilizando. Con `_id: 0` declaramos expresamente que no lo necesitamos para esta consulta.

Romper la cobertura con un cambio intencionado

Prueba una proyección que también pida la descripción:

```
db.ventas.find(
  {clienteId: 809},
  {_id: 0, clienteId: 1, importe: 1, descripcion: 1}
).hint("ix_cobertura_cliente_importe")
 .limit(5)
 .explain("executionStats")
```

El índice no incluye `descripcion`. Para devolver ese campo, MongoDB necesita normalmente acceder a documentos de la colección. Observa cómo cambia `totalDocsExamined`.

Conclusión: un índice cubre una consulta concreta, no una colección completa. No es razonable agregar todos los campos del documento a cada índice solo para intentar obtener cobertura.

---

15. Rango numérico: por qué una condición cambia los candidatos

Piensa en la pantalla de administración que muestra ventas de 100 a 200 euros. No conocemos su número exacto; necesitamos encontrar un conjunto dentro de un intervalo.

Escribe primero la pregunta

```
db.ventas.find({
  importe: {$gte: 100, $lt: 200}
}).limit(5)
```

`$gte` significa «mayor o igual que» y `$lt` «menor que». Por tanto, incluye 100 y excluye 200. Ese detalle es importante en facturas, precios y fechas.

Construye un índice orientado al intervalo

```
db.ventas.createIndex({importe: 1}, {name: "ix_importe_rango"})
```

Después:

```
db.ventas.find({
  importe: {$gte: 100, $lt: 200}
}).hint("ix_importe_rango")
  .limit(5)
  .explain("executionStats")
```

Qué mirar: si hay muchas ventas en ese rango, MongoDB puede examinar numerosas entradas de índice. El índice permite recorrer claves del intervalo, pero no garantiza encontrar «una sola venta» porque la pregunta devuelve potencialmente muchas.

Comparación de interpretación

Cambia el límite de 5 a 50. La selección lógica por importe no cambia, pero el número de resultados que solicitas aumenta. ¿Se examinan más entradas? Compruébalo en `explain()` sin asumir una cifra concreta.

Si todo devuelve cero: inspecciona `db.ventas.findOne()` y comprueba que `importe` es un número. No escribas `"100"` entre comillas si quieres comparar números.

---

16. Campo poco selectivo: aprender cuándo un índice ayuda menos

El campo `estado` tiene pocos valores posibles, como `"pagada"` y `"pendiente"`. Una gran cantidad de ventas puede compartir el mismo estado.

Experimenta con la distribución

```
use curso_indices
db.ventas.countDocuments({estado: "pagada"})
db.ventas.countDocuments({})
```

Divide el primer resultado por el segundo. Si una de cada cuatro ventas aparece como pagada, el filtro devuelve una fracción apreciable de la colección. No es como buscar un número de venta único.

Crear un índice de igualdad

```
db.ventas.createIndex({estado: 1}, {name: "ix_estado_solo"})
```

Mide la consulta:

```
db.ventas.find({estado: "pagada"})
  .hint("ix_estado_solo")
  .explain("executionStats")
```

Puede devolver miles de documentos y examinar muchas claves. A diferencia de la consulta por `numero`, aquí no deberías esperar `totalDocsExamined: 1`.

¿Por qué usamos hint?

Para estudiar el índice concreto. Después repite la misma consulta **sin `hint`** y observa qué plan decide MongoDB. El planificador no está obligado a elegir nuestro índice si otro plan le resulta mejor.

La enseñanza: no basta con afirmar «el campo tiene índice». También importa cuántos documentos satisfacen el filtro y cuál es el volumen de datos solicitado.

---

17. Igualdad más rango en un índice compuesto

El gerente pide ventas de la tienda de Madrid por encima de cierto importe. Vamos a pasar de un solo campo a un diseño combinado.

Formular la consulta

```
db.ventas.find({
  tienda: "Madrid",
  importe: {$gte: 300}
}).limit(5)
```

Identifica qué parte es igualdad y cuál es rango. La primera restringe la tienda; la segunda selecciona un intervalo monetario.

Proponer un índice

```
db.ventas.createIndex(
  {tienda: 1, importe: 1},
  {name: "ix_tienda_importe"}
)
```

Ejecuta la consulta con `.explain("executionStats")` y después añade `.hint("ix_tienda_importe")` antes de `.explain`. Compara plan y contadores.

Por qué no invertiríamos el orden sin analizarlo

La clave `{tienda:1,importe:1}` agrupa primero los valores de una tienda y permite recorrer después los importes correspondientes. Invertir el orden cambia la organización de las entradas. A veces una estructura alternativa puede convenir, pero hay que razonar qué filtros dominan la carga de trabajo.

Prueba de transferencia: elimina la condición de tienda y conserva solo el filtro por importe. El prefijo izquierdo de la clave compuesta ya no se satisface. ¿Deberíamos garantizar el mismo acceso selectivo? No. Compruébalo en el plan.

---

18. Array de objetos y $elemMatch: no unir valores de elementos distintos

Hasta ahora hemos buscado en un array de textos. Algunas ventas tienen otro tipo de lista:

```
productos: [
  {sku: "SKU45", cantidad: 2}
]
```

Cada elemento es un objeto con varios campos. Cuando queremos que **la misma línea de producto** cumpla varios criterios, conviene conocer `$elemMatch`.

Ejemplo de consulta

```
db.ventas.find({
  productos: {
    $elemMatch: {
      sku: "SKU45",
      cantidad: {$gte: 2}
    }
  }
}).limit(5)
```

El operador expresa «existe un elemento del array que cumple ambas condiciones». Sin `$elemMatch`, distintas condiciones de un array pueden satisfacerse sobre elementos distintos según la forma de consulta; esa semántica podría dar un resultado no esperado.

Comprobar antes de indexar

Ejecuta `db.ventas.findOne({productos: {$exists: true}})` y confirma que el campo realmente es una lista de objetos. Después ejecuta la consulta y observa los elementos.

Puedes experimentar con un índice:

```
db.ventas.createIndex(
  {"productos.sku": 1, "productos.cantidad": 1},
  {name: "ix_productos_sku_cantidad"}
)
```

Este diseño indexa rutas dentro de los objetos del array. Debe estudiarse con las restricciones de multikey y con `explain()`; no se debe concluir que indexar cualquier par de arrays distintos está permitido.

Si no aparecen resultados

La condición `sku` puede existir, pero `cantidad` ser insuficiente; reduce temporalmente la cantidad mínima y examina el dato, no cambies el operador sin entenderlo.

---

19. Búsqueda de texto con puntuación: no confundir relevancia con fecha

Una búsqueda textual puede devolver muchas descripciones. MongoDB dispone de una puntuación de texto que ayuda a ordenar resultados según el mecanismo de búsqueda clásico.

La consulta

Asegúrate de que existe el índice `text` en `descripcion`. Después, dentro de `mongosh`:

```
db.ventas.find(
  {$text: {$search: "reclamacion urgente"}},
  {descripcion: 1, puntuacion: {$meta: "textScore"}}
).sort({puntuacion: {$meta: "textScore"}}).limit(5)
```

La salida puede mostrar una puntuación numérica. **No es un porcentaje de acierto** ni una estimación de la gravedad de una reclamación; solo refleja el mecanismo de ranking textual.

Distinguir dos preguntas

- «¿Qué documentos contienen palabras relacionadas con reclamación urgente?». Es una pregunta de texto.
- «¿Cuáles de esos documentos son los más recientes?». Añade una exigencia cronológica que no es equivalente a ordenar por relevancia.

Si la aplicación necesita ambas propiedades, tiene que decidir cómo combinarlas. No supongas que una puntuación de texto reemplaza a un campo `fecha`.

Si aparece un error de índice de texto

Usa `db.ventas.getIndexes()` para verificar que el índice existe. Una colección autogestionada admite, con el mecanismo clásico, un índice `text` por colección aunque pueda cubrir varios campos.

---

20. Comprender qué devuelve una consulta geoespacial

Una localización no es un nombre de ciudad. Para consultar por proximidad, MongoDB necesita puntos con coordenadas coherentes.

Inspeccionar primero un documento

```
use curso_indices
db.ventas.findOne({ubicacion: {$exists: true}})
```

Localiza `ubicacion.type` y `ubicacion.coordinates`. Si ves `"Point"` con dos números, recuerda su orden: **longitud, latitud**.

Ejecutar proximidad sobre el mismo formato

```
db.ventas.find({
  ubicacion: {
    $near: {
      $geometry: {type: "Point", coordinates: [-3.7038, 40.4168]},
      $maxDistance: 5000
    }
  }
}).limit(5)
```

Si el índice `2dsphere` todavía no existe, créalo mediante el ejercicio anterior. Observa si devuelve documentos dentro de la distancia solicitada.

Dos errores habituales

Coordenadas invertidas. Escribir primero latitud y luego longitud cambia el punto, aunque ambos números parezcan «geográficos».

Unidades confundidas. El ejemplo utiliza metros para `$maxDistance` con GeoJSON y `2dsphere`. No interpretes 5000 como kilómetros.

Qué no demuestra la prueba

Que existan cinco resultados no implica que todas las ventas se produzcan a menos de 5 km de ese punto. Hemos pedido solo un máximo de cinco resultados; para contar resultados de una región sería necesaria otra consulta apropiada.

---

21. Partial index: incluir solo ventas pendientes

Una oficina gestiona las ventas con estado `"pendiente"`. Puede querer un índice que incluya fechas de esas ventas, pero no mantener entradas del mismo índice para los pedidos pagados de hace años.

Preparar el índice

Dentro de `mongosh`:

```
use curso_indices
db.ventas.createIndex(
  {fecha: -1},
  {
    name: "ix_pendientes_fecha",
    partialFilterExpression: {estado: "pendiente"}
  }
)
```

Lee la decisión: la clave del índice es `fecha`. La condición `partialFilterExpression` limita qué documentos entran en él. No es una condición que se aplique automáticamente a todas las consultas de la aplicación.

Consulta compatible con la selección

```
db.ventas.find({estado: "pendiente"})
  .sort({fecha: -1})
  .limit(5)
  .explain("executionStats")
```

El planificador puede considerar este índice porque la consulta solicita exactamente el subconjunto que el índice contiene.

Consulta que no puede perder resultados

```
db.ventas.find({})
  .sort({fecha: -1})
  .limit(5)
  .explain("executionStats")
```

Ahora pedimos ventas de **todos** los estados. Si MongoDB solo consultara el índice parcial, omitiría pagadas y enviadas. Por eso el plan no puede utilizar esa estructura como si representara toda la colección.

Razona antes de continuar

Si mañana una venta pasa de pendiente a pagada, ¿por qué debe actualizarse el índice parcial? Porque ya no pertenece al subconjunto indexado. El índice ahorra espacio respecto a indexar todas las fechas, pero no está libre de mantenimiento.

22. Distinguir nombres, claves y opciones del índice

Cuando enseñamos `createIndex`, es fácil pensar que el nombre visible es el índice completo. No lo es. El diseño incluye claves, orden y opciones.

Inspección

```
db.ventas.getIndexes()
```

Localiza el índice `ix_numero`, el compuesto de cliente y fecha y el índice de texto. Para cada uno identifica:

- `name`: nombre usado para referirnos a él.
- `key`: campos y dirección o tipo de índice.
- Opciones como `unique`, `partialFilterExpression` o `expireAfterSeconds`.

Experimento de consistencia

Intenta describir este diseño sin utilizar su nombre:

```
{clienteId: 1, fecha: -1}
```

Una descripción correcta sería: «primero organiza por cliente ascendente y, dentro de cada cliente, por fecha descendente». No basta decir «índice de cliente».

La trampa del nombre

Podríamos poner a un índice el nombre `"indice_rapido"` y seguiría sin ser necesariamente rápido. El nombre es un identificador administrativo, **no una propiedad de rendimiento**.

¿Cómo se relaciona con el laboratorio?

Si una operación con `hint("ix_cliente_fecha")` falla porque no encuentra ese índice, comprueba su nombre real en `getIndexes()`. Evita copiar nombres de otros ejercicios sin inspeccionar primero el estado de tu colección.

---

23. Índice único compuesto: una regla de negocio con dos campos

Puede que una tienda numere sus facturas a partir del 1 cada nuevo año. Si el número se repite en años diferentes, imponer `unique` únicamente sobre `numero` sería incorrecto. La regla auténtica podría ser: *dentro de un mismo año, el número no se repite*.

Vamos a construir una colección independiente para no alterar las ventas del curso.

Paso 1. Reiniciar solo la colección de prueba

```
use curso_indices
db.facturas_demo.drop()
db.facturas_demo.createIndex(
  {anio: 1, numero: 1},
  {unique: true, name: "ix_factura_anio_numero"}
)
```

Paso 2. Insertar dos facturas válidas

```
db.facturas_demo.insertMany([
  {anio: 2025, numero: 101, cliente: "A"},
  {anio: 2026, numero: 101, cliente: "B"}
])
```

La operación debe ser aceptada: aunque se repite el número, cambia el año. La clave **compuesta** es diferente.

Paso 3. Intentar repetir la combinación

```
db.facturas_demo.insertOne({
  anio: 2025, numero: 101, cliente: "C"
})
```

Esperamos un error de duplicado `E11000`. Comprueba:

```
db.facturas_demo.countDocuments({})
```

Debe mostrar 2.

Qué has aprendido: una restricción de unicidad se aplica a la clave definida, que puede contener más de un campo. En sistemas reales, la clave única debe expresar fielmente la regla de negocio.

---

24. TTL con una fecha límite: interpretar el tiempo sin confundir unidades

Ya vimos cómo caduca un documento. Ahora comprobaremos por qué almacenar un campo como fecha real es distinto de almacenar su representación como texto.

Dos documentos de prueba

```
use curso_indices
db.caducidad_demo.drop()
db.caducidad_demo.createIndex(
  {vence: 1},
  {expireAfterSeconds: 0}
)

db.caducidad_demo.insertMany([
  {_id: "fecha", vence: new Date(Date.now() - 60000)},
  {_id: "texto", vence: "2020-01-01"}
])
```

El primero contiene una fecha vencida. El segundo contiene un texto que **se parece** a una fecha, pero no tiene el tipo adecuado para la política TTL.

Observación

```
db.caducidad_demo.find()
```

Al actuar el monitor TTL, el documento con `vence` de tipo fecha puede desaparecer. El de texto no debe considerarse una fecha válida simplemente por su apariencia.

El peligro de diseñar una regla de seguridad con TTL

Aunque la eliminación funcione, puede tener retraso. Una aplicación que permita acceso hasta que desaparezca físicamente un documento podría mantener activa una sesión ya caducada durante cierto intervalo.

Conclusión obligatoria: el monitor TTL facilita la limpieza de datos; las reglas de autorización y caducidad lógica deben evaluarse por separado cuando la seguridad lo requiera.

---

25. Ocultar un índice: experimentar sin destruirlo

Imagina que hemos mantenido un índice durante meses pero creemos que ya no se usa. En lugar de eliminarlo de inmediato, MongoDB permite hacerlo invisible al planificador.

Preparar un índice que sabemos que existe

```
use curso_indices
db.ventas.createIndex(
  {clienteId: 1, fecha: -1},
  {name: "ix_cliente_fecha"}
)
```

Observar el plan normal

```
db.ventas.find({clienteId: 809})
  .sort({fecha: -1})
  .limit(5)
  .explain("executionStats")
```

Anota el índice escogido si aparece en el plan.

Ocultarlo y repetir

```
db.ventas.hideIndex("ix_cliente_fecha")
```

Repite la misma consulta y observa si MongoDB utiliza otro índice o cambia el trabajo realizado. **El resultado puede depender de los índices alternativos que tengas.**

Restaurar siempre

```
db.ventas.unhideIndex("ix_cliente_fecha")
```

Comprueba: `getIndexes()` te permite observar el estado del índice. No abandones este laboratorio con un índice oculto accidentalmente.

Interpretación

El índice oculto sigue ocupando espacio y sigue manteniéndose con las escrituras. Ocultarlo no equivale a borrarlo. Tampoco puede forzarse un índice oculto mediante `hint`. Esta técnica ayuda a investigar el efecto de una posible retirada.

---

26. $indexStats: observar uso, no adivinar popularidad

Un índice puede haber sido creado por una tarea que se ejecuta solo una vez al mes. ¿Qué ocurre si estudiamos su uso durante los diez minutos de clase? Tal vez aparezca sin accesos, pero eso no demostraría que sea innecesario.

Consultar los contadores

```
use curso_indices
db.ventas.aggregate([{$indexStats: {}}])
```

Cada fila resume información de un índice y sus accesos observados en el nodo. Los campos pueden incluir nombre y estadísticas de acceso.

Generar una consulta de prueba

```
db.ventas.find({numero: 199998}).limit(1).toArray()
```

Vuelve a ejecutar `$indexStats` y busca el índice sobre `numero`. Puede reflejar actividad, dependiendo del plan y de las características del servidor.

Lo que no debemos afirmar

Un contador de cero **no prueba** que el índice carezca de valor para la aplicación. La consulta de negocio puede no haberse ejecutado en la ventana de observación; también pueden existir varias instancias o cambios que reinicien contadores.

Ejercicio de criterio profesional

Elabora una recomendación de dos frases: «Antes de eliminar este índice necesitamos observar... y comprobar...». Una buena respuesta menciona el periodo real de uso, las consultas que lo emplean y el efecto esperado sobre escrituras y lecturas.

---

27. Tamaño de índices: no confundir documentos con estructuras auxiliares

Una tienda puede almacenar cientos de miles de ventas y varios índices. Necesitamos separar cuánto ocupan los documentos y cuánto ocupan las estructuras de acceso.

Consultar estadísticas

```
use curso_indices
db.ventas.stats()
```

La salida puede incluir campos como `count`, `size`, `storageSize`, `totalIndexSize` e `indexSizes`. Las cifras exactas dependen de la compresión y de las características del motor.

Interpretar una tabla

Anota:

Dato	¿Qué representa?
`count`	Documentos de la colección
`size`	Tamaño lógico aproximado de los datos
`storageSize`	Espacio de almacenamiento asociado a la colección
`totalIndexSize`	Espacio total de los índices
`indexSizes`	Desglose por índice

Relacionar tamaño con el diseño

Un índice con muchas claves o que indexa arrays puede tener costes diferentes a uno pequeño sobre campos con pocos valores. Pero no es suficiente afirmar que «multikey siempre ocupa el doble». Hay que medir.

Pregunta: si eliminas un índice que se usa para búsquedas importantes, ¿qué ganas y qué arriesgas? Puedes reducir recursos de mantenimiento y almacenamiento, pero empeorar consultas que dependían de él.

---

28. allPlansExecution: por qué no siempre gana el plan que imaginabas

MongoDB puede considerar varias estrategias. El modo `executionStats` describe la ejecución del plan ganador, pero existe otro modo que aporta información sobre la evaluación de planes candidatos.

Ejecutar una consulta con candidatos

```
db.ventas.find({clienteId: 809})
  .sort({fecha: -1})
  .limit(5)
  .explain("allPlansExecution")
```

Busca los apartados relacionados con el plan ganador y los planes considerados.

Cómo leer sin ahogarse en el JSON

Empieza por comprobar cuántos resultados espera la consulta y qué índices existen. Solo después explora los planes candidatos. No trates todos los campos de la salida como si fueran imprescindibles para un principiante.

Qué no significa este modo

No garantiza haber ejecutado completamente cada candidato hasta devolver todos los resultados: cierta información corresponde a la evaluación durante la selección. No confundas esas estadísticas parciales con las de una ejecución normal completa.

Ejercicio de comprensión

¿Por qué un índice nuevo podría no ser elegido? Porque otro ya satisface mejor la consulta, porque el filtro es poco selectivo o por características concretas del plan. La presencia de un índice en `getIndexes()` no obliga al optimizador a utilizarlo.

---

29. Cambiar una sola variable: la base de un experimento válido

En varias prácticas comparamos consultas antes y después de un índice. Vamos a comprobar por qué debemos conservar iguales las condiciones cuando queremos atribuir una diferencia a una causa.

Primera medición

```
db.ventas.find({numero: 199998})
  .explain("executionStats")
```

Anota `nReturned` y `totalDocsExamined`.

Segunda medición: un cambio deliberadamente incorrecto

```
db.ventas.find({estado: "pagada"})
  .explain("executionStats")
```

Es otra consulta, que puede devolver miles de documentos. Si comparas sus tiempos con los de la primera, no puedes concluir nada serio sobre el efecto del índice `numero`.

Tercera medición: volver a un experimento correcto

Repite la primera consulta después de un cambio controlado en su índice, sin alterar el filtro ni los documentos de la colección. Solo entonces tendrás una comparación significativa del efecto de ese cambio.

Conclusión escrita

«Una medición de rendimiento solo permite explicar diferencias si controlamos lo que cambia. Si modifico a la vez filtro, tamaño de colección e índices, no puedo atribuir la diferencia a uno de ellos».

---

30. Escritura e índices: qué tiene que mantener el servidor

Un índice mejora determinadas búsquedas, pero también debe actualizarse al cambiar los documentos. Lo demostraremos con una colección aislada, sin millones de registros.

Crear la colección de ensayo

```
use curso_indices
db.escrituras_demo.drop()
db.escrituras_demo.createIndex({codigo: 1})
db.escrituras_demo.createIndex({categoria: 1, precio: 1})
```

Insertar y actualizar

```
db.escrituras_demo.insertOne({
  codigo: "L-01", categoria: "libros", precio: 19.50
})
db.escrituras_demo.updateOne(
  {codigo: "L-01"},
  {$set: {precio: 24.50}}
)
db.escrituras_demo.findOne({codigo: "L-01"})
```

El documento cambia de precio. MongoDB debe mantener la correspondencia entre el dato nuevo y los índices afectados. Este experimento **no demuestra por sí solo una pérdida de velocidad medible**; muestra una obligación de consistencia.

Pregunta para razonar

Si una colección se actualiza constantemente, ¿conviene agregar veinte índices que nadie consulta? Probablemente no. Cada índice adicional puede exigir mantenimiento sin aportar beneficio al patrón real de lecturas.

---

31. Diagnosticar un índice que aparentemente «no funciona»

Un compañero afirma: «He creado un índice, pero la consulta sigue siendo lenta». Antes de borrar datos, sigue esta investigación.

Pregunta 1. ¿Cuál es la consulta exacta?

Pide el filtro completo, la ordenación, la proyección y el límite. No sirve «busco por cliente».

Pregunta 2. ¿Existe el índice que afirma?

```
db.ventas.getIndexes()
```

Verifica campos y orden. Un índice `{clienteId:1,fecha:-1}` no es un índice `{fecha:-1,clienteId:1}`.

Pregunta 3. ¿Cómo se ejecutó realmente?

```
db.ventas.find({clienteId: 809})
  .sort({fecha: -1})
  .limit(5)
  .explain("executionStats")
```

Localiza plan y contadores. No interpretes «tardó más» sin otros datos.

Pregunta 4. ¿Hay un problema de tipos o de selección?

Comprueba el tipo del campo, la cardinalidad y la cantidad de resultados. Un índice sobre un campo que devuelve casi toda la colección puede no tener el efecto esperado.

Pregunta 5. ¿Qué hipótesis probaríamos después?

Proponer un índice alternativo tiene sentido **después** de entender por qué el actual no cumple los requisitos. Si no sabes explicar el problema, aún no es momento de crear otro.

---

32. Caso integrador: la librería cambia sus pantallas

El responsable de la librería plantea tres necesidades:

- localizar una venta concreta por número;
- mostrar las diez últimas ventas de un cliente;
- buscar descripciones que mencionen una reclamación.

Preparación

Trabaja sobre la colección `curso_indices.ventas`. Comprueba que contiene ventas, que conoces el tipo de `numero` y `fecha`, y que la colección no fue reiniciada accidentalmente.

Parte A. Proponer y justificar

Escribe qué índice utilizarías para cada consulta. Debe haber tres explicaciones diferentes: igualdad muy selectiva, filtro y ordenación, búsqueda de términos.

Parte B. Ejecutar

```
db.ventas.find({numero: 199998}).explain("executionStats")

db.ventas.find({clienteId: 809})
  .sort({fecha: -1})
  .limit(10)
  .explain("executionStats")

db.ventas.find({$text: {$search: "reclamacion"}})
  .limit(10)
  .explain("executionStats")
```

Parte C. Elaborar un pequeño informe técnico

Para cada consulta, registra el resultado, el índice seleccionado, los documentos examinados y qué explicación ofrecerías a alguien que no conoce MongoDB. Si aparece un plan inesperado, documenta el hecho: no inventes resultados para que coincidan con tu hipótesis.

Parte D. Revisión crítica

¿Podría un solo índice cubrir de manera óptima las tres consultas? No deberías dar por sentado que sí. Cada pregunta tiene un patrón diferente y la colección también necesita aceptar escrituras.

Criterio de éxito: no es una captura de pantalla con todos los comandos en verde. Es ser capaz de demostrar que sabes formular la necesidad, escoger un índice candidato, observar el plan real e interpretar el coste.

Resolución de incidencias frecuentes

«docker: command not found». No estás ejecutando Docker desde una terminal que tenga disponible el programa; revisa instalación y ruta.

«Cannot connect to Docker daemon». Inicia Docker Desktop/Engine y comprueba su estado.

«Port is already allocated». El puerto local asignado ya está ocupado; detén la instancia que lo utiliza o cambia el mapeo externo en `compose.yaml`.

«required variable MONGO_ADMIN_PASSWORD is missing». No existe `.env` en la raíz o carece de esa variable; crea el archivo desde la plantilla y proporciona una contraseña de prueba.

«La colección no contiene 200000 ventas». Comprueba `db.getName()`, vuelve a ejecutar la carga y espera a que termine. No sigas con índices sobre un conjunto vacío.

«02_antes.js dice que ya existen índices». La comparación debe partir de la colección reinicializada; ejecuta primero `01_cargar_ventas.js`.

«No veo IXSCAN». Comprueba el nombre del campo, el tipo del valor, el índice existente y el plan completo. Si hay otros índices, el planificador puede elegirlos.

«Ya se declaró const sinIndice». La sesión de `mongosh` ya contiene esa variable. Usa otro nombre o abre una nueva sesión.

«Error E11000». En la práctica de unicidad es la respuesta esperada: confirma que la base de datos ha rechazado un duplicado.

«TTL no borra inmediatamente». Es asíncrono. Comprueba el tipo fecha, el índice y la actividad del monitor; no presupongas borrado instantáneo.

Criterio de aprendizaje

La práctica está realmente aprendida cuando puedes explicar, sin leer el comando, qué consulta se formuló, cuál era el estado inicial de los datos, qué cambió con el índice y qué evidencias lo prueban. Si no puedes responder a esas preguntas, repite el experimento con menos datos y analiza cada etapa.

Referencia

MongoDB 8.0 — Índices · explain()